Quantum Information Processing (2024) 23:349
https://doi.org/10.1007/s11128-024-04559-8

Quantum space-efficient large language models for Prolog query translation

Roshan Ahmed[1] · **S. Sridevi**[2]

Received: 4 May 2024 / Accepted: 25 September 2024 / Published online: 16 October 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

As large language models (LLMs) continue to expand in complexity, their size follows an exponential increase following Moore's law. However, implementing such complex tasks with LLMs poses a significant challenge, as classical computers may lack the necessary space to run or store the model parameters. In this context leveraging the principles of hybrid quantum machine learning for language models offers a promising solution to mitigate this issue by reducing storage space for model parameters. Although pure quantum language models have demonstrated success in recent past, they are constrained by limited features and availability. In this research we propose the DeepKet model an approach with a quantum embedding layer, which utilizes the Hilbert space generated by quantum entanglement to store feature vectors, leading to a significant reduction in size. The experimental analysis evaluates the performance of open-source pre-trained models like Microsoft Phi and CodeGen, specifically fine-tuned for generating Prolog code for geo-spatial data retrieval. Furthermore, it investigates the effectiveness of quantum DeepKet embedding layers by comparing them with the total parameter count of traditional models.

Keywords Word2Vec · Large language model · Generative AI · Quantum computing · Quantum machine learning · Transfer learning · Prolog

✉ S. Sridevi
sridevi.s@vit.ac.in

Roshan Ahmed
roshan.ahmed2122002@gmail.com

[1] Department of AI & Robotics, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India

[2] School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India

1 Introduction

Large language models (LLMs) are a subcategory of AI that involves deep learning to create a language model that is capable of understanding the context in text. The core of LLMs involves having a neural network to process information, along with a special type of algorithm called self-attention [1]. A dedicated attention layer picks and chooses the actively relevant tokens in a sentence. Each token in the sentence is assigned an attention score that tells the encoder–decoder model how much importance is to be given to each token.

This type of model, primarily known as a transformer model, allows for more context information to be stored as the model continues through the sequence. Common uses for transformers are in machine translation, summarizing, semantic understanding and sentiment analysis. LLMs are transformer models that have a significant amount of data fed through them, typically a large amount of data from internet archives of messages, conversations and code [2].

1.1 The need for Prolog code generation

LLMs frequently fall victim to an issue called "Hallucination", where an LLM forcibly generates an answer to a question that it may not have an answer to. A method of solving the issue of hallucination is to integrate a knowledge base with the LLM [3]. The LLM is trained not to answer the question directly, but instead to search a knowledge base for listed information, and use that to answer the relevant question. Lang-Chain [4] is one of the modules that allow this type of architecture. For example, when using code-generative LLMs, it is possible to generate an SQL query and link it to an SQL database to get and retrieve the answer from there.

Prolog [5] is a query language that shows a lot of potential in AI-related use cases. It is used to query, search and deduce information using facts and relations through first-order logic. It parses the query step-by-step through the knowledge base which makes it good at logical information retrieval while keeping it free from hallucination that most LLMs are victims of. By incorporating Prolog into LLMs, it is possible to reduce hallucinations in LLMs. The goal is to see which open-source models can be fine-tuned for Prolog translation and to see the potential parameter reduction possible using quantum computing aspects. Open-source models, typically available through hugging face, are available for free use and for fine-tuning, this is one of the criteria of models shortlisted to be used for fine-tuning an LLM to write Prolog code.

This fine-tuning, however, is very time-consuming due to the billions of trainable parameters in a single model. PEFT or parameter-efficient fine-tuning is a method of fine-tuning LLMs by only having to train a few select parameters. This is done by inserting low-ranked layers called adapters between other layers and only training those layers. This method is called LoRA for low-rank adapters. This allows fine-tuning only certain use cases without altering the original model, which may deteriorate its performance. These low-rank adapters can be quantized into low-bit values. This algorithm is called QLoRA or quantized low-rank adapter pace [6].

1.2 The increasing in sizes of LLMs

Since LLMs have a large amount of data to deal with, the number of parameters required to handle the data is also large. Large language models increase in parameter count as the years go by [7]. ELMo in 2018 had 94 million parameters, BERT in late 2018 had 340 million parameters, and GPT-2 finally broke the one billion line and reached 1.5 billion parameters in 2019. GPT-3, made popular by OpenAI's Chat GPT, sits at 175 billion parameters that was out in mid-2020.

Mid-2021 saw Megatron-Turing NLG hitting the 500 billion line at 530 billion parameters. As of the literature review of this research, GPT-4 in 2023 has one of the largest numbers of parameters at 1.76 trillion parameters and costs more than 100 million US Dollars for training. There is an observation similar to Moore's law [8] where the number of parameters of cutting-edge LLMs follows an exponential growth.

Due to the rapidly growing speed of parameters, there is an urge to look into space-efficient LLMs. A method of reducing the size is using quantization of parameters by storing parameters as low-bit integers, typically 8 bits, and then multiplying them by a parameter scaling to get real values float parameters. Another counterintuitive alternative is to look into low-parameter layers and architecture to lower the overall parameter count. Microsoft Phi2 [9], for example, is a small-size model that performs as good as models nearly 400% its size.

1.3 The quantum solution for space-efficiency

Quantum computing is the usage of particles such as electrons or rays of light to represent data to create superposition states and use it for computations [10] . Superposition is the state in which a quantum bit can be written as a probabilistic combination of multiple states. This superposition state can be altered to store and compute exponentially large amounts of data per quantum bit. Feats like encryption and energy state optimizations can be accomplished using this state.

A potential solution for this lies in quantum computing. Quantum computing succeeds in dimensionality reduction during computation. A binary variable of length n bits requires n bits or units of space for storage and computation. However, n quantum bits a.k.a qubits are capable of storing and computing 2^n numerical values in parallel. Take for example 1024 bits of data. A classical computer requires 1024 bits of storage space. A quantum computer will require only $\log_2(1024)$ or 10 qubits for its representation.

Although fully quantum versions of language models exist [11], they are not researched and developed to the extent of currently available classical models [1]. Hybrid models are an option that most quantum implementation methods utilize, where some aspect of an algorithm runs on a quantum computer and a subroutine runs on a quantum computer.

[1] For the purpose of this research work, "Classical" refers to any algorithm, method, model or architecture that does not have any quantum aspect in it, e.g., "Classical Computer" is a normal computer.

1.3.1 Existing methods of compression

The primary method of lowering the size of layers is in compositional learning [12]. Most compositional methods are capable of lowering the size by 94–99%, but it is possible to compress further by using quantum super-positioning. Pure quantum solutions exist [13], but there are no hybrid solutions available for classical-quantum models. Preliminary work done by Word2Ket researchers [14] shows it is possible to go lower using the Hilbert space, but their system works on only on purely classical hardware and has no quantum implementation. Implementing the Hilbert Space system on quantum systems is the goal of this paper, where we have obtained lossless compression rate of 99.97%.

1.4 The goal of deepKet

DeepKet is the proposed research work for creating hybrid language models. Creating a transferable embedding layer that reduces a significant portion of a language model allows the user of the model to train and implement DeepKet to reduce the size of any classical language model. DeepKet has shown to have great outcomes in reducing over 100k parameters in a 64-feature dimension text sorting model to about a thousand. This research work attempts to test the extent of DeepKet to test its ability to match high-parameter language models such as LLMs which tend to have about a thousand feature dimensions [15].

One common layer that nearly all language models share is the word embedding layer. Word embeddings are a real-valued feature vector that represents the meaning of a word in a context. The higher-dimensional space created by word embedding vectors is called the semantic space. It is possible to replace the embedding layer matrix with a deep neural network that composes the embedding layer in real-time and stores only the compositional code [16]. Another key method, Word2Ket [14] uses the Kronecker product, which is a quantum computing concept to compute word embedding by using lower-dimensional tensors. This reduces the sizes of embedding layers from hundreds of thousands to barely a few hundred.

All quantum machines exist in a probabilistic combination of quantum states. A probabilistic combination of quantum states exists in the Hilbert space, similar to how the combination of word embedding vectors exists in the semantic space [17].

Each qubit is represented as a ket, which represents a quantum state, for example, $|0>$ or $|1>$. The probability values of the different quantum states were represented as a matrix called a bra $< p_0|$ or $< p_1|$ and together made the bra-ket notation $< p_0|0>$. The Hilbert space can store this data which is how quantum computing can allow for computing higher-dimensional problems by using very few qubits.

Variational quantum circuits (VQCs) [18] are circuits that deal with quantum bits instead of 0 s and 1 s. They allow for exponentially more dimensions of data to be stored. Pennylane is a Python module that facilitates the creation and use of VQCs through APIs like Qiskit and Regetti (Fig. 1).

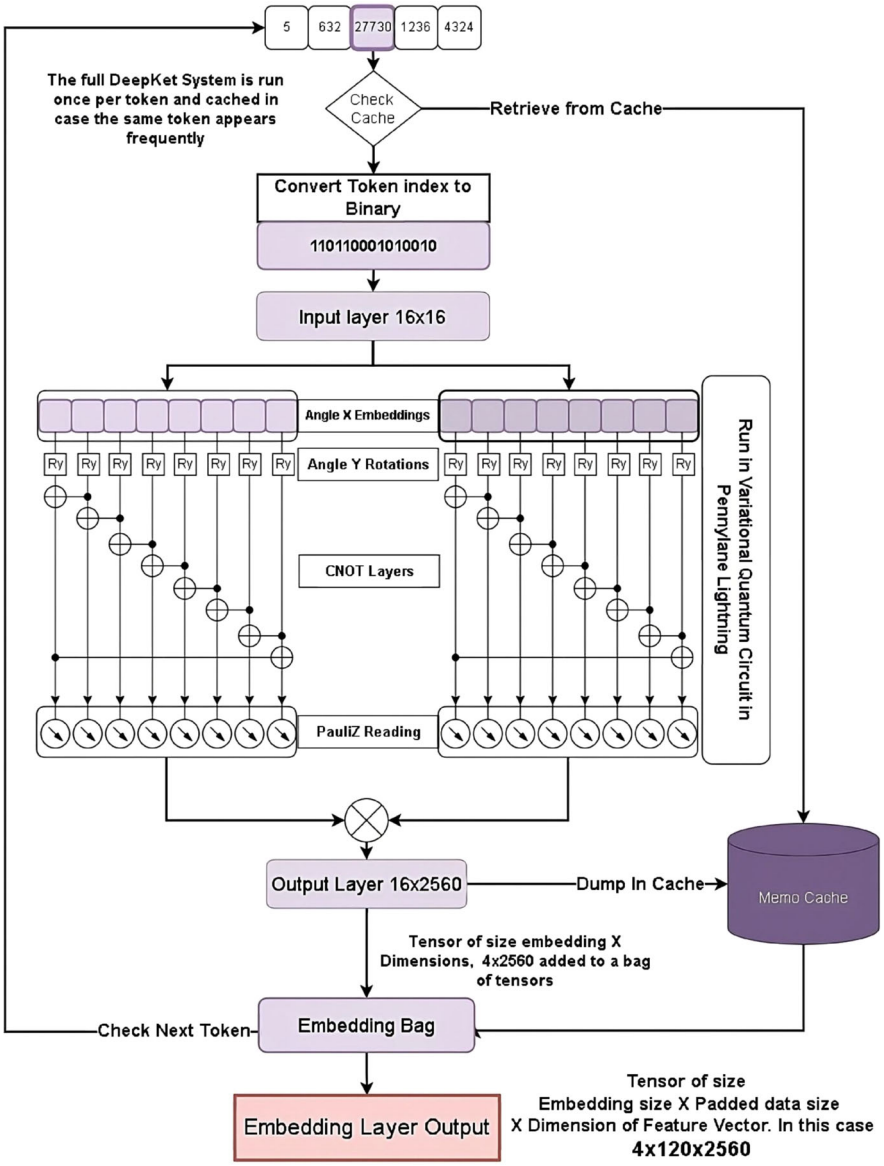


Fig. 1 DeepKet implementation model architecture

2 Dataset creation

2.1 Prolog language

Prolog [5] is a knowledge-handling language that uses first-order logic to resolve queries based on facts and conditions. Prolog is a logical programming language

that is used for creating knowledge bases. Prolog uses first-order logic to parse and extract data from a database of rules and facts. Logical programming is different from standard databases and programming because it is capable of making decisions and parsing logical statements, which most normal languages cannot do inherently unless specified during runtime.

According to the literature survey, the fine-tuned model in this research work is a pioneer in writing open-source Prolog code using LLMs. One of the goals is to find the best open-source model among CodeT5 [19] models, Microsoft Phi [9] and CodeGen [15] that can be fine-tuned to properly write Prolog code.

Prolog functions by creating facts and rules.

- Facts: Facts are single atomic terms of data. For example, "object A is red" or "person A is from Canada" are atomic terms of data.
- Rules: Rules are logical statements that are applied during runtime. For example, "If the object is made of wood and rectangular it is a door" or "If a person is from Canada they are not a native citizen"

The full advantage of Prolog happens due to the ability to chain multiple logical statements. During runtime, the Prolog takes a single query and parses each layer of logical statements in a depth-first search parsing every statement and fact along the way. An example of this is given below:

```
brown(object1)
rectangular(object1)
hard(object1)

large(object2)
rectangular(object2)

wood(A): - brown(A), hard(A)
door(A): - rectangular(A), wood(A)
```

Parsing the above statement. Door(A) parses into rectangular(A) and wood(A). First checking for all rectangular objects, both object1 and object2 return true; thus, the second condition is checked.

Parsing wood(A) requires parsing brown(A) and hard(A). Object1 passes parsing brown and rectangular, but object2 fails parsing brown as that term does not exist for it. This object1 is made of wood and object2 may not be. Going up one more level, the parser concludes that object1 is made of wood and object2 may not be a door.

2.2 GeoQuery dataset for fine-tuning

Geo-spatial technology is described as the field of mapping and analysis of locations on Earth. It remains an active field of research and is frequently updated. This means that the tools required to store and retrieve this data are also frequently improving due to its relevancy in technological advancements. The SPIRIT project [20] was created to make a distinguished database to store such data, and the STEWARD project [21] looked into creating a text search engine-based retrieval system.

Table 1 Sample question-query dataset

Question	Prolog Query
give me the largest state	A, largest (A, state(A))
give me the number of rivers in California ?	A, count (B, (river(B), loc(B,C), const(C, stateid(California))), A)
how big is Alaska?	A, (size(B,A), const(B, stateid(alaska))
how many major cities are there ?	A, count(B, (major(B), city(B)), A)

Geo-spatial data have also made its way into the domain of Prolog. GeoData project [22] established a database of 1000 Prolog facts about American geo-spatial data. The data consist primarily of population, area, proximity, and river-oriented data. In an attempt to create a parser for this dataset, a dataset of queries and text-based questions was also created [23]. There are two sizes for this dataset, geoquery210 and geoquery880 based on the complexity of the questions. For fine-tuning in this research work, geoquery880 is used. An example of the dataset is given in Table 1

Geoquery880 is a list of 880 query-text pairs stored as a special file type meant for the parser the original authors were working on, which could not be read normally, so RegEx was used to extract, store and separate the prompts and queries. Split and rejoin were done to remove unnecessary symbols and punctuation such as periods or spaces.

2.3 Prompt-response tokens for deepKet

For the sake of faster and more dedicated training, the DeepKet was not trained on all tokens. The original draft of the model was trained on all tokens, but with over 50k tokens and only one epoch taking a few hours for one of the larger models, a different dataset was created and utilized.

Since the goal of the research work is experimenting with code writing, tokens with code-relevant strings like brackets, padding and syntax get higher priority and need to be trained more than occasionally once in a while variable names, or tokens unrelated to the data. A prompt generator was set up that created an input–prompt–output string that contains the important words that will be used by the classical model. Input, prompt and output text is then tokenized as per the requirements of the model and then used for student–teacher learning.

An sample instance of a prompt–response string is given below:

```
Instruct: Convert this into a Prolog query.
Which state contains the most rivers?
Output:
A,most(A,B,(state(A),loc(B,A),river(B)))
```

Both the classical original model and the quantum DeepKet model are given the same prompt-response string. The differences between the output feature vectors are taken as loss values for training. An example of the different outputs is shown in Figs. 5, 6 and 7

3 Methodology

3.1 Hilbert space

Hilbert space [17] is an Euclidean vector space that allows the generalization of methods in linear algebra. Any system can be represented and modeled as a Hilbert space, and any vector in the Hilbert space represents an energy state of the system. When a system is in superposition, the current state vector does not correspond to any pure state vectors, instead, is represented as a mixed state vector or a linear combination of pure state vectors.

As an example, a single-particle quantum bit where the state of the system is the state of a bit is zero or one, represented by $|0>$ or $|1>$, and the Hilbert space can be represented as $p_0|0> + p_1|1>$ where p_0 and p_1 represent the probability of each state. In quantum notation, a pure state vector is represented as $|\phi>$ called a ket, while the probability of each state is represented as $<\phi|$ called a bra. Together they form the bra-ket notation. This bra-ket notation can form the Hilbert space, with a single point representing the superposition state of a system. This superposition state can be manipulated to do generalized linear calculations in the Hilbert space, which forms the essence of quantum computing. Both bras and kets are represented as vectors, where ket is a pure state vector and bra is a mixed state vector, the inner product of which gives the Hilbert space written as $<\phi|\psi>$.

3.2 Equation explanation

Hilbert space can be created and represented using the following representation:

$$<\phi| \otimes |\psi> \tag{1}$$

When a measurement is taken, the Hilbert space is measured along the projection of $<\phi|$.

$$H := |\phi><\phi|\psi> \tag{2}$$

Thus, if the created Hilbert space $|\psi>$ is stored in the circuit, it can be measured along with any state tensor, which is used to store the index of the token.

Quantum states can be split as well.

$$|\psi_{\text{dimension,token}}> = |\psi_{\text{dimension}}> \otimes |\psi_{\text{token}}> \tag{3}$$

So, by dedicating a few bits to the index and a few bits to the dimension index, a Kronecker product can be created that stores the full embedding matrix. The full measurement becomes as follows:

$$H := (|\psi_{\text{dimension}} > \otimes < \psi_{\text{dimension}}|\phi_{\text{dimension}} >) \otimes |\phi_{\text{token}} > \tag{4}$$

Variational quantum circuits can implement this equation in a natural implementation; this is the key purpose of choosing quantum circuits over classical algorithms since the Kronecker product is a natural outcome of quantum computation. In the VQC required to replace a linear layer in a deep neural network, a rotation by an axis followed by measuring it along another axis is done

$$H := T \bigotimes_{k=1}^{2} \left(R_x(z_k \pi/2) \right) |\phi_{\text{token}} > \tag{5}$$

3.3 DeepKet

DeepKet is a method of storing tensors within a quantum network that can be extracted by inputting the index in binary. Quantum machine learning networks are capable of storing significantly larger amounts of data in the same number of parameters or storage space. DeepKet is a hybrid network that involves a classical input and output layer, and two parallel quantum circuits.

The standard state of a quantum circuit is a maximally mixed state where all states have an equal probability of occurrence. However, when subject to external factors, such as being shot with beams of light or subject to an electric pulse, the probability changes. The goal of DeepKet is to create a quantum system that stores feature vectors in the Hilbert space that stores the that can then be token-wise extracted.

There is an advantage that can be utilized in terms of states. By distributing a state with higher qubits into a state with lower qubits, the inner product can be taken to recreate the larger state. This is similar to converting $|b_{0011} >$ into $|b_{00} > \otimes |b_{11} >$. This act converts the larger Hilbert space into smaller Hilbert spaces. Assume these smaller Hilbert spaces to be left state and right state. The right state is used to represent the binary format of the token index, and the left side represents the feature index. For example, $|24 > \otimes |64 >$ is used to extract the 24th feature of the 64th token.

3.3.1 Variational quantum circuits

State creation, superposition and retrieval are done by using a variational quantum circuit (VQC). Pennylane follows the Bloch sphere model, where states of a single qubit can be represented as a tensor that lies on a sphere and can be rotated using VQC gates. A qubit can be rotated or measured on an axis, which forms the basis of quantum circuits. Additionally, entangler layers affect adjacent wires or qubits which allows for superposition states.

DeepKet VQC starts by taking input and rotating it about the X-axis. This forms the complex conjugate of the input quantum state, or the right state as mentioned above. The entangler layer performs rotations on the Y-axis, as well as CNOT gates

that enable entanglement. The final layer, the measurement layer performs a Pauli-Z function that measures the value across the Z axis, finally giving that as an output.

The number of qubits or wires required for this circuit depends on $\log_2(n)$ where n is the number of input tokens. Multiple testing by the original DeepKet experiments shows that a certain number of qubits and layers slightly lower than $\log_2(n)$ show decent results with reduced training time, while going above it gives diminishing returns concerning runtime.

3.3.2 Experimental implementation in simulator

We have executed the quantum computations in the fast Pennlylane Lightning statevector simulator to estimate the training and testing compression embeddings. This Pennylane statevector simulator returns the final quantum statevector of the executed quantum circuit regarding the circuit behavior. Hence, it is used for all kinds of experimental analysis in this proposed research. Statevector simulator is a flexible tool for experimental investigation as it allows researchers and developers to examine and fully understand the functioning of quantum circuits, enhance circuits, and get insightful knowledge about the quantum states involved in the computations. In every experimental result analysis, we considered having the number of shots as 500, which is one of the important quantum hyperparameters. Each shot comprises an individual circuit execution that yields a measurement response. The crucial role of the shot number is apparent in its effect on the precision and dependability of the computed results. Hilbert space being a probabilistic matrix means that having more shots will make the measured probability match the theoretical probability, while additionally increasing the precision of the values. However, after the difference between measured and theoretical values reaches a low consistent value, adding more shots increases the runtime and costs relative to the number of shots. Thus, an ideal number of shots are estimated.

3.3.3 DeepKet architecture

The output layer of the DeepKet model is a linear layer. The purpose of this linear layer is to resize the output to fit the interaction point of the embedding layer and the rest of the model. Additionally, the linear layer functions similarly to a weighted sum of the inputs, making it similar to an inner product between two concatenated tensors, which is where the implementation of the Kronecker product exists.

The input layer also has a linear layer but the reason for this is different from the output layer. There exists a bug in Pennylane where taking binary inputs directly causes all outputs to become reset every time, despite the inputs being different. Replacing binary inputs with floats or activation layers like ReLU did not solve this bug; thus, the suggested method of using a linear layer was implemented, resolving this issue.

The hyperparameters used for a 8-wire 1-layer DeepKet model created for the Microsoft Phi2 model along with their parameter count are listed in Table 2

Table 2 Layers of an 8-wire, 1-layer, 2560-output DeepKet embedding layer

Layer type	Hyperparameters	Count of trainable parameters
Input formatting layer	Linear (16,16)	272
Quantum circuit	2x VQC (8 wires, 1 layer)	2x 16
Output formatting layer	Linear (16, 2560)	43,820

4 Training implementation

4.1 Models for fine-tuning

There are five models used and compared here: Microsoft Phi2, which is a multi-purpose LLM, and four different models from the CodeGen series.

CodeGen 2 B and 350 M are the sizes of the models, both trained on the same datasets, referring to the number of parameters, being 2 billion and 350 million, respectively

Within them, there are two variants being used. CodeGen NL series is a non-fine-tuned version of the CodeGen LLM that has only been tuned on normal text. CodeGen Multi series is trained in a few programming languages such as Python, Java and C++.

There is another variant of CodeGen called CodeGen mono which has not been used for the sake of this research work. Mono is a variant that is trained only on Python. It was discarded as the Python-biased LLM may make it harder to fine-tune for a different language.

The list of models and their hugging face tags used are: Microsoft Phi2 [9], CodeGen-350 M-nl, CodeGen-350 M-multi, CodeGen-2 B-nl,CodeGen-2 B-multi [15]. All models have been tested and compared after training over 10 epochs (documented in Fig. 3), and the results are documented in Table 3.

4.2 DeepKet architecture definition

DeepKet has been implemented using Pennylane and PyTorch. The goal of the model is to take a binary input and convert it into a tensor as an output that is normalized and usable as a feature vector.

The circuit used for DeepKet runs on lightning qubit which is one of the simulators available on Pennylane. Four and 8 qubit circuits with 1 or 2 layers were all tested and compared. Additionally, there is a memorization feature where frequently used tokens are cached for fast retrieval. When training using brute force, the cache overflowed and caused errors, after which the feature was disabled or limited. During the brute force training of every token, it was disabled, but when using the geoquery880 prompt-response dataset, it was found that the memorization feature reduces runtime for each batch from 5–10 min down to 40–50 s.

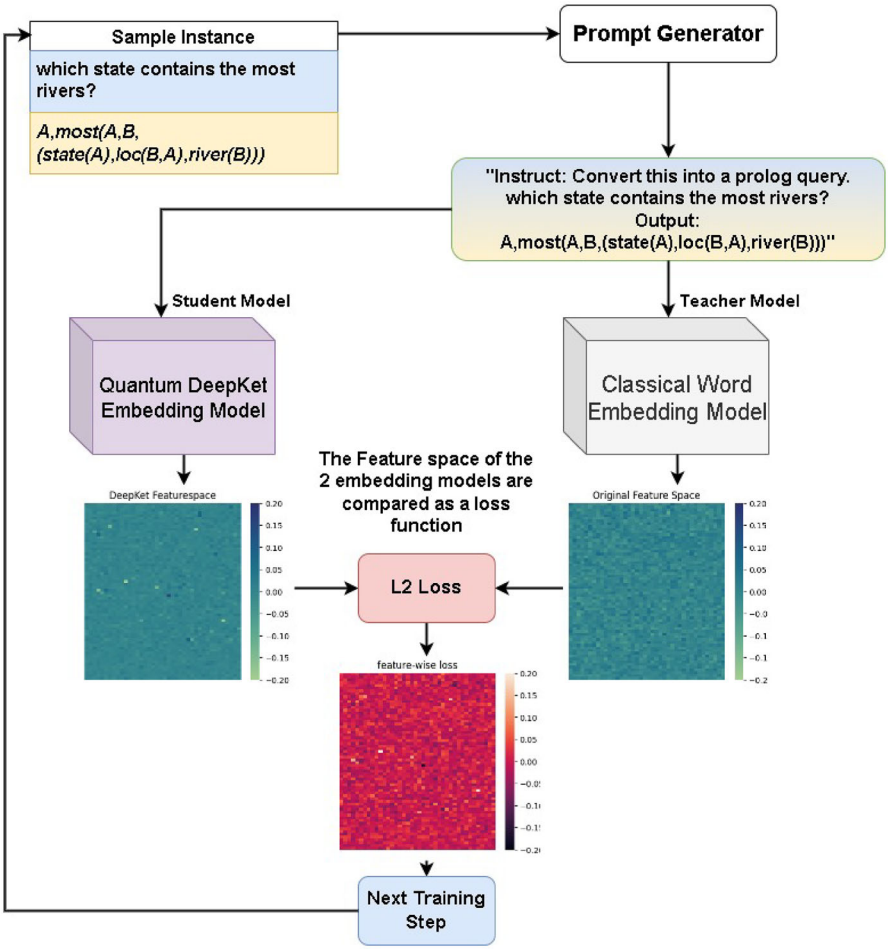


Fig. 2 DeepKet student–teacher learning workflow

4.3 Training optimization algorithms for deepKet

Training for the DeepKet and the fine-tuning were both done in Kaggle cloud computing notebooks which are accelerated with a P100 CPU. The cloud computing notebooks have their own limitations which need to be resolved before proceeding.

4.3.1 Federated learning

Kaggle cloud computing notebooks have a limited session time of 9 h. However, the training time for the planned 5 epochs cannot be completed in a single session as some models' pre-optimization took very close to the 9-hour deadline and sometimes over the time duration which causes the cloud computer to dump the session along with

the active model and data being trained. To bypass this limitation and for reliable and safer training sessions, federated learning techniques were involved.

A single main database holding all training checkpoints was created. During the initialization of the session, a method would check for the latest version of the model available and import the checkpoint state. if no checkpoint is available, a new checkpoint file is created. Once the planned training for a session is completed, the session will be closed, returning the model checkpoints to the main database and updating it. Then, subject to device availability, the next session would be opened repeating the process.

The sessions created by the federated learning algorithm were kept under 4–5 h to have a few hours of safety net to create backups of the model in case of time overflow.

4.3.2 Student teacher learning

The draft workflow tested was designed to be trained on all tokens at least once per epoch, and special epochs twice this puts the dataset to be over 100 k data points. Since each data point individually triggers and LLM and a VQC, The training time for a single epoch would go above 100–200 training hours. Thus, other methods needed to be utilized.

Instead of training for all tokens, the model was trained only for relevant tokens that are seen in the dataset. for example tokens for emotions, food and people would not be seen in the geoquery880 dataset and thus need not be trained. This was done by concatenating the prompt and the response data points into a prompt-response string that is then tokenized and batched.

The feature-wise loss and the comparison between the student and teacher can be seen in Fig. 6. The workflow for this student–teacher training methodology is represented in Fig. 2

5 Results

Models have been compared to find the best working model to fine-tune for prompt-to-query conversion. The suitable models that have shown positive results have been further used to replace the embedding layer and compare the sizes. The reduction in size from a hundred million to 50 k parameters shows great compression potential for other layers. Figure 3 visualizes what models are compatible for fine-tuning the Prolog model along with the details of parameter count and reduction in Table 3.

For the research experiments, about 200 device hours were used over multiple devices and sessions. The device hours used for fine-tuning LLMs were relatively small, being about a total of 30 h.

Hugging face was used to run the models, along with other experimentation such as transfer learning, student–teacher running and other training runs using PyTorch. Pennylane along with the Lightning Qubit simulator were used for running the variational quantum circuit.

Table 3 Model comparison

Model	Embedding parameters	DeepKet parameters	Loss	Comments
Microsoft Phi	131 M	43.8 k	0.141	Properly terminates reply and correct syntax
CodeGen-350 M-nl	52.4 M	43.8 k	0.148	Incorrect syntax and extra tokens
CodeGen-350 M-multi	52.4 M	43.8 k	0.129	Correct syntax and extra tokens
CodeGen-2 B-nl	131 M	43.8 k	0.130	Parses query as a question and attempts to answer it
CodeGen-2 B-multi	131 M	43.8 k	0.127	Properly terminates and correct syntax

5.1 Model shortlisting analysis

Fine-tuning was done using the LoRA module and 8-bit QLoRA modules [6]. The training was done for 5 epochs over the whole dataset using a paged Adam 8-bit optimizer and 1000 steps.

LoRA was targeted at the output projection and linear layers of all models used, and the language model head of the CodeGen family of models. All models were trained on a cloud notebook based on Kaggle, and a GPU P100 accelerator on the geoquery880 dataset [24].

All models have been extracted from hugging face. The prompt–query data points have been tested and compared after training over 10 epochs (documented in Fig. 3), and the results are documented in Table 3.

CodeGen family of LLMs tends to produce extra words and does not have an automatic end-generation token. Thus, directly plugging CodeGen family LLMs into an output or Prolog parser may show up with a few syntax issues. The only model here that properly terminates its sentence generation is Microsoft Phi2.

CodeGen's NL 2B model does not parse the prompt as a query, instead parses it as a question and attempts to answer it directly. For example, in the test case "What states share a boundary with New York", the model outputs a list of American states, instead of returning a Prolog query. Thus, they have been seen as unsuitable for the next step. The 2B multi-model occasionally gives wrong syntax but is otherwise stable.

From CodeGen's 350 M, they produce Prolog queries as expected, but with the regular CodeGen issue of extra tokens. However, the NL model does not produce the correct output, returning faulty statements that do not match the syntax for Prolog queries. Thus, the 350 M is also seen as partially unsuitable.

The exception to the above case is the CodeGen-350 M-multi. CodeGen has fewer parameters and thus stops it from parsing prompts as questions. Not being fine-tuned

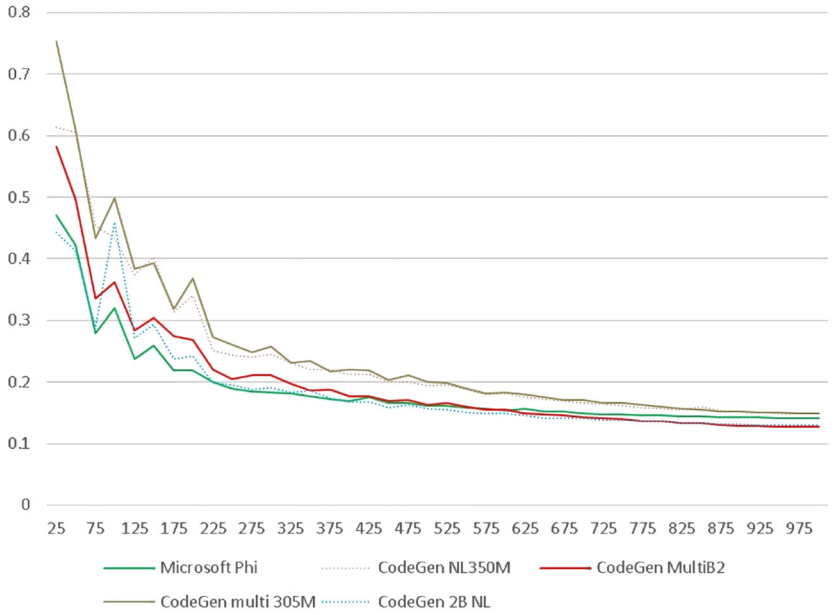


Fig. 3 Validation loss of different models. Dotted models are rejected

on specific code, it does not mistake syntaxes from one language to the other and causes syntax errors and thus is a proper working base model.

Microsoft Phi2 produces proper statements in syntax and has also passed multiple other test cases with the proper query. Additionally, it is the only model that terminates its token generation at the right point.

Thus, out of the tested models, Microsoft Phi2 (hugging face tag: Microsoft-phi2) was shortlisted as it gave the most stable and terminated outputs consistently over 5 test cases. CodeGen's multi-series models tended to output the wrong syntax in 1–2 test cases out of 5 tested.

5.2 DeepKet result analysis

The training for this model was done on a cloud computing notebook hosted by Kaggle, with a GPU P100 as an accelerator for the classical components and (lightning.qubit) simulator for the quantum parts. Even with the above accelerators, the overall involvement of quantum simulators massively slows down the training speed. Without quantum accelerators, a single epoch will take nearly 34 h to train, with the quantum accelerator, the speed increases to 5 h per epoch.

The size of the embedding layer gives the maximum possible efficiency in relative reduction of size. The different models and the size of their embedding parameters are given in Table 3. The smaller models have a better embedding-to-normal layer ratio, focusing on smaller models will increase the overall relative efficiency, but to lower an objective size, it is better to focus on larger models.

Table 4 Time optimization steps and their extrapolated run times for 5 epochs compared in the 8-wire 1-layer model.

Variants of deepKet tested	Approximate time taken
Current (optimized)	6–8 h
No memorization, geoquery880	25–27 h
Brute force dataset	30–35 h
No memorization, brute force	100–120 h
4-1 model (smaller model)	4–5 h
No lightning qubit	200 + hours

Time is taken by extrapolating the fluctuating times from the first few iterations when the model was too long and had to be terminated

5.2.1 Speed optimization

The deployed tactics for model optimization were tested over multiple attempts. Each attempt was done on a truncated, smaller version of the model with only 1% of the data. The only purpose of this test was to optimize and see if a model optimization strategy was worth deploying for a full training session. Every time a single iteration took over a few minutes to run, the single iteration time was noted and the model training was terminated. This way, the limited amount of resources was saved to test and optimize the model before training the model completely for 5 epochs. After all the tests were done, the final training process was started. All the variants and strategies tested are found in Table 4.

5.2.2 Choice of performance metric

The hybrid classical-quantum neural network requires significant computational time, and our current resources are insufficient to fully train the model and compare results in terms of regular performance metrics such as accuracy. Due to the extensive simulation time required, obtaining accuracy metrics as a method of model comparison is not feasible, since the DeepKet model cannot be fully trained. With better access to computational resources and quantum computers, we could complete the training process, allowing for a reasonable comparison of accuracy metrics. To measure the performance of the current stage of our implementation, we considered feature space loss values as performance metrics instead of accuracy. Feature space loss has been further elaborated in the upcoming section. However, preliminary testing on a partially trained DeepKet model showed an approximately 8% drop in accuracy for a model compressed by 99%.

5.3 Feature space loss analysis

Feature space is the raw normalized output values from the embedding layer taken into the fine-tuned LLMs. In the visualization, The output feature vector is reshaped into a 2d array for better visual comparability. Looking into the feature space at different

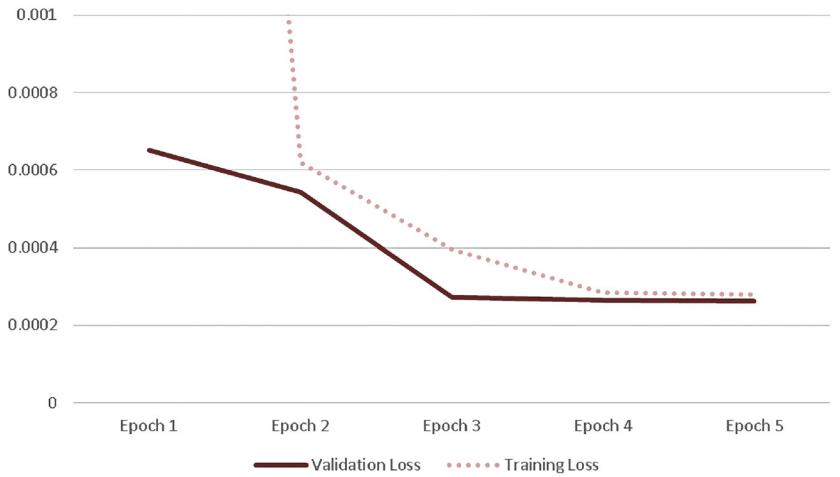


Fig. 4 DeepKet implementation training loss for geoquery880 prompt dataset

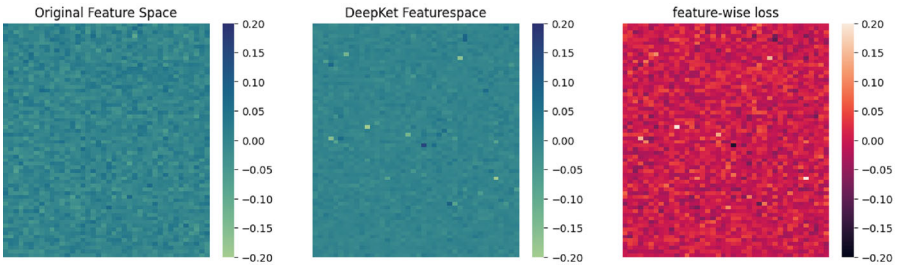


Fig. 5 Feature space of untrained token or token not in dataset

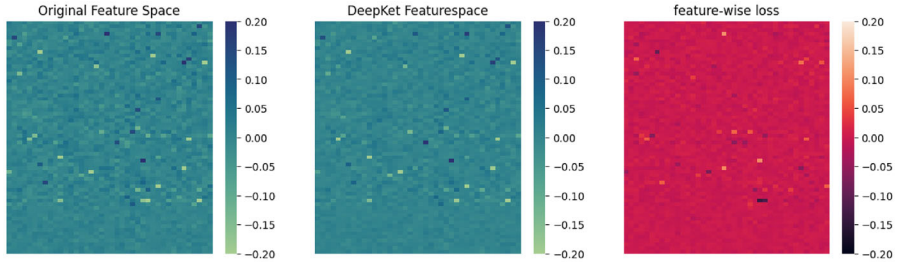


Fig. 6 Feature space of occasionally occurring token

training levels between the default embedding layer and the DeepKet embedding layer shows the progress of training. Additionally, the loss visualization being smoother and redder means the model is getting better and matching the default embedding layer

As seen in Fig. 7, DeepKet works perfectly under properly trained circumstances. However, training that for every token requires a lot of computational power. Training the models halfway, similar to partially trained Fig. 6, could work for less important

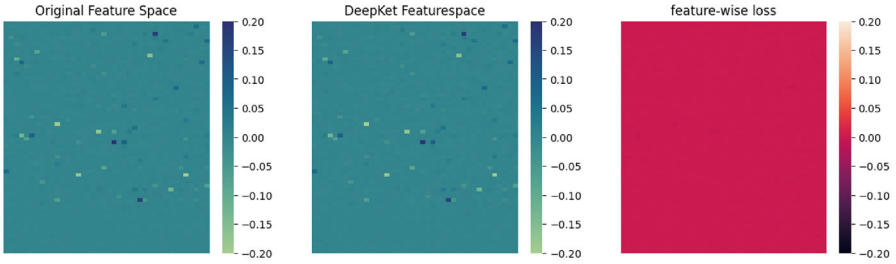


Fig. 7 Feature space of commonly occurring token for fully trained model

cases like classification, but for fully open usage, most tokens will remain in Fig. 5 state (Fig. 4).

In the untrained Fig. 5, visually there are a lot of mismatches between the features. The dimensions that give the context for Prolog query that we have trained on are not very prominent in the original feature space. When implementing for larger use cases and in better hardware, all tokens must be trained, but for this fine-tuned case, only relevant and frequently occurring tokens have been trained.

As seen in partially trained Fig. 6, there are a lot of matches between the different key features. As an example, a lot of the features less than -0.2 in value are correctly identified. However, due to the nature of normalization, the amplitudes are affected and are still off by an error of about 0.1. This shows that the correct context features are fine-tuned, and the prominent feature dimensions show that they represent the context of this specific use case.

Finally, in the fully trained Fig. 7, it is clear that for a properly trained token, it is possible for the DeepKet model to recreate a feature vector properly and is the core proof of concept that it is viable but requires proper and longer training sessions in better hardware

6 Conclusion

Models Microsoft Phi, Codegen NL 350 M and 2B, and Codegen Multi 350 M and 2B were tested to write a Prolog query based on the geoquery880 dataset for American geographical data. Microsoft Phi showed to be the best at being fine-tuned and was thus shortlisted for the next phase.

The embedding layer of Microsoft phi is taken and used to train a DeepKet model to match the outputs. Tokens that occur frequently are aptly tuned showing that it is possible to represent the output word embedding using DeepKet.

Due to unavailable access to quantum computers, the models were trained on a simulation of a quantum computer, and all times and learning rates were based on the simulations, the same may not apply to a real quantum computer, and the efficiency must be verified. LLM models used for sentiment analysis have shown good results, but the final model obtained after embedding replacement for code generation does not show optimal results. Further training and optimization must be done.

Further goals for the research work are running the same experiment on a better dataset and training DeepKet on all available tokens to create a more flexible embedding layer, such as for chat bots or summarizing tasks where the range of tokens generally used is wider. Access to better devices, both quantum and classical, will allow for further research than just this proof of concept for a relatively small dataset compared to what LLMs are expected to be trained on.

Author Contributions Author Roshan.A have contributed to the development of the proposed model, came up with the experimental results, and wrote the manuscript. Author Sridevi.S have reviewed the manuscript and improved the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. Advances in neural information processing systems **30** (2017)
2. Kim, K., Ghatpande, S., Kim, D., Zhou, X., Liu, K., Bissyandé, T.F., Klein, J., Le Traon, Y.: Big code search: a bibliography. ACM Comput. Surv. **56**(1), 1–49 (2023)
3. Reshmi, S., Balakrishnan, K.: Implementation of an inquisitive chatbot for database supported knowledge bases. Sādhanā **41**, 1173–1178 (2016)
4. Pandya, K., Holia, M.: Automating customer service using langchain: Building custom open-source gpt chatbot for organizations. arXiv preprint arXiv:2310.05421 (2023)
5. Colmerauer, A.: An introduction to prolog iii. Commun. ACM **33**(7), 69–90 (1990)
6. Raschka, S.: Finetuning LLMs Efficiently with Adapters (2023)
7. Liu, Y., Han, T., Ma, S., Zhang, J., Yang, Y., Tian, J., He, H., Li, A., He, M., Liu, Z., et al.: Summary of chatgpt-related research and perspective towards the future of large language models. Meta Radiol., 100017 (2023)
8. Simon, J.: Large language models: A new Moore's law. HuggingFace. https://huggingface.co/blog/large-language-models (2021)
9. Javaheripi, M., Bubeck, S., Abdin, M., Aneja, J., Bubeck, S., Mendes, C.C.T., Chen, W., Del Giorno, A., Eldan, R., Gopi, S., et al.: Phi-2: The surprising power of small language models. Microsoft Research Blog (2023)
10. Cirac, J.I., Perez-Garcia, D., Schuch, N., Verstraete, F.: Matrix product states and projected entangled pair states: concepts, symmetries, theorems. Rev. Modern Phys. **93**(4), 045003 (2021)
11. Chen, Y., Pan, Y., Dong, D.: Quantum language model with entanglement embedding for question answering. IEEE Trans. Cybern. **53**(6), 3467–3478 (2021)
12. Shu, R., Nakayama, H.: Compressing word embeddings via deep compositional code learning. arXiv preprint arXiv:1711.01068 (2017)
13. Cantone, D., Faro, S., Pavone, A., Viola, C.: Quantum circuits for fixed matching substring problems. In: Science and Information Conference, pp. 667–686 (2024). Springer
14. Panahi, A., Saeedi, S., Arodz, T.: word2ket: Space-efficient word embeddings inspired by quantum entanglement. arXiv preprint arXiv:1911.04975 (2019)
15. Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C.: CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis (2023)
16. Shu, R., Nakayama, H.: Compressing word embeddings via deep compositional code learning. arXiv preprint arXiv:1711.01068 (2017)
17. Schuld, M., Killoran, N.: Quantum machine learning in feature Hilbert spaces. Phys. rev. lett. **122**(4), 040504 (2019)

18. Ramezani, S.B., Sommers, A., Manchukonda, H.K., Rahimi, S., Amirlatifi, A.: Machine learning algorithms in quantum computing: A survey. In: 2020 International Joint Conference on Neural Networks (IJCNN), pp. 1–8 (2020). IEEE
19. Wang, Y., Wang, W., Joty, S., Hoi, S.C.H.: CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation (2021)
20. Jones, C.B., Purves, R., Ruas, A., Sanderson, M., Sester, M., Van Kreveld, M., Weibel, R.: Spatial information retrieval and geographical ontologies an overview of the spirit project. In: Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 387–388 (2002)
21. Lieberman, M.D., Samet, H., Sankaranarayanan, J., Sperling, J.: Steward: architecture of a spatio-textual search engine. In: Proceedings of the 15th Annual ACM International Symposium on Advances in Geographic Information Systems, pp. 1–8 (2007)
22. Tang, L.R., Mooney, R.J.: Using multiple clause constructors in inductive logic programming for semantic parsing. In: European Conference on Machine Learning, pp. 466–477 (2001). Springer
23. Sánchez Solís, J.P., González-Barbosa, J.J., Solís, J.F., Sánchez-Hernández, J.P.: Two new challenging resources to evaluate natural language interfaces to databases generated based on geobase and geoquery. Inst. Ing. Tecnol. (2020)
24. Punjani, D., Singh, K., Both, A., Koubarakis, M., Angelidis, I., Bereta, K., Beris, T., Bilidas, D., Ioannidis, T., Karalis, N.: Template-based question answering over linked geospatial data. In: Proceedings of the 12th Workshop on Geographic Information Retrieval, pp. 1–10 (2018)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.